BEYOND THE BLACK BOX: DISENTANGLED AND CONTROLLABLE GENERATIVE MODELS FOR MEDICAL IMAGING

AXEL MARTIN ROMERO ANTOLIN

PhD in Artificial Intelligence
Facultat d'Informàtica de Barcelona (FIB)
Universitat Politècnica de Catalunya (UPC)

January 2029 – version 4.2

Axel Martin Romero Antolin: *Beyond the Black Box: Disentangled and Controllable Generative Models for Medical Imaging*, PhD in Artificial Intelligence, © January 2029

SUPERVISORS:
Prof. Javier Béjar Alonso

LOCATION:
Barcelona, Spain

TIME FRAME:
January 2029

ABSTRACT

Short summary of the contents in English... a great guide by Kent Beck how to write good abstracts can be found here:

https://plg.uwaterloo.ca/~migod/research/beckOOPSLA.html

RESUM

Aquí anirà el abstract en català...

We have seen that computer programming is an art,
because it applies accumulated knowledge to the world,
because it requires skill and ingenuity, and especially
because it produces objects of beauty.

— knuth:1974 [knuth:1974]

ACKNOWLEDGMENTS

Put your acknowledgments here.

Many thanks to everybody who already sent me a postcard!

Regarding the typography and other help, many thanks go to Marco Kuhlmann, Philipp Lehman, Lothar Schlesier, Jim Young, Lorenzo Pantieri and Enrico Gregorio[1], Jörg Sommer, Joachim Köstler, Daniel Gottschlag, Denis Aydin, Paride Legovini, Steffen Prochnow, Nicolas Repp, Hinrich Harms, Roland Winkler, Jörg Weber, Henri Menke, Claus Lahiri, Clemens Niederberger, Stefano Bragaglia, Jörn Hees, and the whole LaTeX-community for support, ideas and some great software.

Regarding LyX: The LyX port was intially done by *Nicholas Mariette* in March 2009 and continued by *Ivo Pletikosić* in 2011. Thank you very much for your work and for the contributions to the original style.

1 Members of GuIT (Gruppo Italiano Utilizzatori di TeX e LaTeX)

CONTENTS

LIST OF FIGURES

LIST OF TABLES

NOTATION

- **Scalars** are denoted by lowercase letters, e.g. $x \in \mathbb{R}$.
- **Vectors** are denoted by bold lowercase letters, e.g. $\mathbf{x} \in \mathbb{R}^n$.
- **Matrices** are denoted by bold uppercase letters, e.g. $\mathbf{X} \in \mathbb{R}^{m \times n}$.
- **Sets** are denoted by calligraphic uppercase letters, e.g. $\mathcal{X}$.
- $\mathbf{X}^\top$ denotes the transpose of a matrix $\mathbf{X}$.
- $\mathbf{I}$ denotes the identity matrix.

- $\mathbb{E}_{\mathbf{x}\sim p}[f(\mathbf{x})]$ denotes the expected value of a function $f(\mathbf{x})$ with respect to a probability distribution p over $\mathbf{x}$.
- $\mathrm{Var}_{\mathbf{x}\sim p}[f(\mathbf{x})]$ denotes the variance of a function $f(\mathbf{x})$ with respect to a probability distribution p over $\mathbf{x}$.
- $\nabla_\theta f(\theta)$ denotes the gradient of a function $f(\theta)$ with respect to the parameters θ.

1 INTRODUCTION

Introduction.

Part I

FOUNDATIONS AND LITERATURE REVIEW

2 DEEP GENERATIVE MODELS

Let's start with the definition of our dataset $\mathcal{D}$, consisting of $N \geqslant 1$ data points, where the instances are assumed to be independently and identically distributed. Assume $\mathbf{x}$ represents the set of observed variables (images in our case), which is a random sample from an unknown underlying process, whose true distribution $p^*(\mathbf{x})$ is unknown. The idea is to approximate the real process with a model $p_\theta(\mathbf{x})$ with parameters θ. Learning has the aim to find a value of parameters θ such that the probability distribution of the model, $p_\theta(\mathbf{x})$, approximates the true distribution for any observed set of variables, i.e. $p_\theta(\mathbf{x}) \approx p^*(\mathbf{x})$.

Deep Generative Models (DGMs) are neural networks, which are a flexible way to parameterise distributions, that learn $p_\theta(\mathbf{x})$, which is commonly referred to as a generative model. Once the model is available, we can generate new samples using sampling methods and compute the likelihood of any given data point $\mathbf{x}'$ by evaluating $p_\theta(\mathbf{x}')$.

2.1 TRAINING OF DGM

The general idea is to learn the parameters θ by minimizing the discrepancy between real and predicted data distribution:

$$\theta^* = \arg\min_\theta \mathcal{D}(p_{data}, p_\theta).$$

Because p_{data} is unknown, we have to choose a discrepancy function that allows efficient estimation from samples from p_{data}.

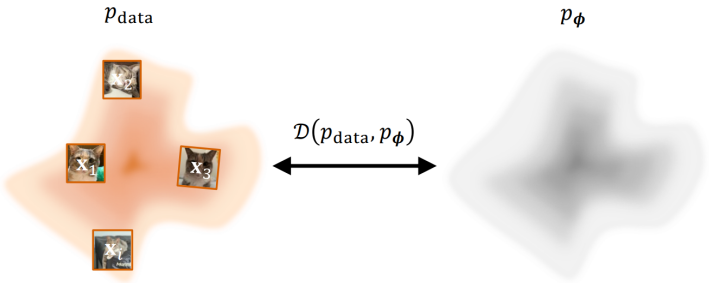


Figure 1: Visualization of the training of DGMs where we minimize a discrepancy measure between the real and unknown data distribution and the approximated one, using a set of i.i.d. samples.[16]

The most common choice is the Kullback-Leibler divergence defined as

$$\begin{aligned} \mathcal{D}_{KL}(p_{data}\|p_\theta) = \int p_{data}(\mathbf{x}) \log \frac{p_{data}(\mathbf{x})}{p_\theta(\mathbf{x})} d\mathbf{x} = \\ \mathbb{E}_{\mathbf{x}\sim p_{data}} \left[\log p_{data}(\mathbf{x}) - \log p_\theta(\mathbf{x})\right], \end{aligned} \tag{1}$$

where, taking into account only the part that depends on the parameter θ, minimizing the KL divergence is equivalent to performing MLE:

$$\min_\theta \mathcal{D}_{KL}(p_{data}\|p_\theta) \iff \max_\theta \mathbb{E}_{\mathbf{x}\sim p_{data}} \left[\log p_\theta(\mathbf{x})\right].$$

In practice, we do not have access to the data distribution, only to some i.i.d. samples $\{\mathbf{x}^{(i)}\}_{i=1}^N \sim p_{data}$, which are used to obtain the empirical estimation of the MSE

$$\mathcal{L}_{MLE}(\theta) = -\frac{1}{N}\sum_{i=1}^{N} \log p_\theta(\mathbf{x}^{(i)}).$$

There are some challenges in the use of neural networks to parameterize the probability distribution p_{data}. We can define a model that produces a real scalar $E_\theta(\mathbf{x}) \in \mathbb{R}$ for input $\mathbf{x}$. To interpret the output as a valid probability density, it has to satisfy the following conditions:

- **Non-Negativity**: $p_\theta(\mathbf{x}) \geqslant 0$ for all $\mathbf{x}$ in the domain. The standard positive function used to guarantee non-negativity is the exponential function, which produces an unnormalized density $\tilde{p}_\theta(\mathbf{x}) = \exp(E_\theta(\mathbf{x}))$.
- **Normalization**: The integral over the entire domain must equal one. To create a valid probability density, we must divide it by its integral over the entire space

  $$p_\theta(\mathbf{x}) = \frac{\tilde{p}_\theta(\mathbf{x})}{Z(\theta)},$$

  where the denominator is the partition function defined as

  $$Z(\theta) = \int \tilde{p}_\theta(\mathbf{x}')d\mathbf{x}'.$$

The partition function introduces a computational challenge in the learning of probabilistic models, especially for high-dimensional problems. This intractability is a central problem that motivates the development of many different families of deep generative models, which will be explained in the following chapters.

3

VARIATIONAL AUTOENCODERS

The core initial idea for the understanding of the Variational Autoencoders (VAEs) is the concept of **Autoencoders** (AEs). This are a type of model that contains two elements, the encoder and the decoder. The encoder is a function that maps the input data $\mathbf{x} \in \mathbb{R}^d$ to a low-dimensional representation $\mathbf{z} \in \mathbb{R}^k$, which is usually much more smaller than the input dimension, $k << d$. This representations live in a space with a more compact and interpretable structure, called the latent space. Then, the decoder is a function that maps the latent representation $\mathbf{z}$ back to the input space, trying to reconstruct the original data point $\mathbf{x}$.
Once we have an autoencoder model trained, a natural question is how to use it as a generative model. One idea is to randomly sample a point $\mathbf{z}$ from the latent space and then pass it through the decoder to get a synthetic data point $\mathbf{x}'$. This method will not produce realistic samples because the latent space is disorganized and irregular. Another option is to encode and image and sample neighboring points in the latent space that could generate similar samples, but the reality is that the output are not meaningful samples. This shows how poorly structured the latent space is. The aim of VAEs is to transform the latent space of an autoencoder into a well-structured space that follows a prior distribution, which allows us to sample from it and generate realistic samples.

It is also important to introduce the concept of latent variable models. This type of variables are part of the model but are not observed in the data, usually denoted as $\mathbf{z}$, which are the hidden factors of variation. The goal of latent variable models is to learn the joint distribution $p_\theta(\mathbf{x}, \mathbf{z})$ of the observed data $\mathbf{x}$ and the latent variables $\mathbf{z}$. The marginal likelihood, model evidence or marginal distribution over the observed data is given by:

$$p_\theta(\mathbf{x}) = \int p_\theta(\mathbf{x}, \mathbf{z}) d\mathbf{z}. \tag{2}$$

When we use neural networks to parameterize the latent variable models we call them **Deep Latent Variable Models** (DLVMs). The simple and most common DLVM is given by the following factorization:

$$p_\theta(\mathbf{x}, \mathbf{z}) = p(\mathbf{z}) p_\theta(\mathbf{x}|\mathbf{z}), \tag{3}$$

where $p(\mathbf{z})$ is the *prior distribution* over the latent variables and $p_\theta(\mathbf{x}|\mathbf{z})$ is the likelihood of the data given the latent variables, also called the

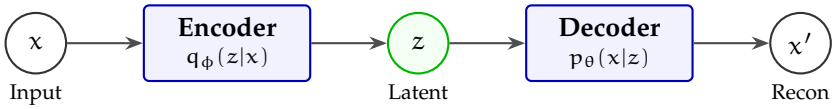


Figure 2: Simplified Variational Autoencoder (VAE) architecture mapping an input x to a latent representation z, and decoding it to a reconstruction x'.

generator distribution. The joint distribution $p_\theta(\mathbf{x}, \mathbf{z})$ is related through the Bayes' rule to the following posterior distribution:

$$p_\theta(\mathbf{z}|\mathbf{x}) = \frac{p_\theta(\mathbf{x}|\mathbf{z})p(\mathbf{z})}{p_\theta(\mathbf{x})}, \tag{4}$$

where we have an intractability problem due to the marginal likelihood $p_\theta(\mathbf{x})$ which is given by an integral over the latent variables (eq 2). To overcome this problem we can use variational inference (VI) which is a technique to approximate the posterior distribution $p_\theta(\mathbf{z}|\mathbf{x})$ with a simpler distribution $q_\phi(\mathbf{z}|\mathbf{x})$ that is parameterized by ϕ.

This is the main idea behind **Variational Autoencoders** (VAEs) [13] which are a type of DLVMs that use VI to learn the latent variable model. It defines a parametric inference model $q_\phi(\mathbf{z}|\mathbf{x})$, also called an *encoder*, where ϕ are the parameters. The idea is to optimize the parameters such that:

$$q_\phi(\mathbf{z}|\mathbf{x}) \approx p_\theta(\mathbf{z}|\mathbf{x}). \tag{5}$$

This VAE formulation allows us to transform an autoencoder model with an unstructured latent space into a generative model with a latent space that follows a prior distribution, typically a Gaussian distribution $p_\theta(\mathbf{z}) = \mathcal{N}(\mathbf{z}; \mathbf{0}, \mathbf{I})$, which is easy to sample from.

3.1 TRAINING OF VAES

For the training of the VAEs we can not directly optimize the marginal likelihood $p_\theta(\mathbf{x})$ due to the intractability problem, but we can optimize a lower bound of the marginal likelihood called the **Evidence Lower Bound** (ELBO). Let's derive the log-likelihood of the data

point $\mathbf{x}$ and see how we can get the ELBO using the Jensen's inequality [12]:

$$\log p_\theta(\mathbf{x}) = \log \int p_\theta(\mathbf{x}, \mathbf{z}) d\mathbf{z} \tag{6}$$

$$= \log \int q_\phi(\mathbf{z}|\mathbf{x}) \frac{p_\theta(\mathbf{x}, \mathbf{z})}{q_\phi(\mathbf{z}|\mathbf{x})} d\mathbf{z} \tag{7}$$

$$= \log \mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} \left[\frac{p_\theta(\mathbf{x}, \mathbf{z})}{q_\phi(\mathbf{z}|\mathbf{x})} \right] \tag{8}$$

$$\geqslant \mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} \left[\log \frac{p_\theta(\mathbf{x}, \mathbf{z})}{q_\phi(\mathbf{z}|\mathbf{x})} \right] \tag{9}$$

$$= \mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} \left[\log \frac{p_\theta(\mathbf{x}|\mathbf{z}) p(\mathbf{z})}{q_\phi(\mathbf{z}|\mathbf{x})} \right] \tag{10}$$

$$= \mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} \left[\log \frac{p_\theta(\mathbf{x}|\mathbf{z})}{\frac{q_\phi(\mathbf{z}|\mathbf{x})}{p(\mathbf{z})}} \right] \tag{11}$$

$$= \mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} \left[\log p_\theta(\mathbf{x}|\mathbf{z}) - \log \frac{q_\phi(\mathbf{z}|\mathbf{x})}{p(\mathbf{z})} \right] \tag{12}$$

$$= \underbrace{\mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} \left[\log p_\theta(\mathbf{x}|\mathbf{z}) \right]}_{\text{Reconstruction}} - \underbrace{\mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} \left[\log \frac{q_\phi(\mathbf{z}|\mathbf{x})}{p(\mathbf{z})} \right]}_{\text{KL Divergence}} \tag{13}$$

We can see that for any data point $\mathbf{x}$, the log-likelihood satisfies:

$$\log p_\theta(\mathbf{x}) \geqslant \mathcal{L}_{\text{ELBO}}(\theta, \phi; \mathbf{x}) \tag{14}$$

$$= \mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} \left[\log p_\theta(\mathbf{x}|\mathbf{z}) \right] - \mathcal{D}_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) || p(\mathbf{z})). \tag{15}$$

If we maximize the previous loss with respect to the parameters θ and ϕ, we are optimizing:

- **Reconstruction**: the first term encourage to maximize the likelihood $p_\theta(\mathbf{x}|\mathbf{z})$, which corresponds to the reconstruction of the data point $\mathbf{x}$ from the latent variables $\mathbf{z}$. Optimizing this term alone risks memorizing the training data, so we need extra regularization.
- **Latent KL Regularization**: the second term encourages the encoder distribution $q_\phi(\mathbf{z}|\mathbf{x})$ to be close to the prior distribution $p(\mathbf{z})$, which is easy to sample from.

Once we have the definition of the training objective, we can use stochastic gradient descent (SGD) to perform joint optimization w.r.t. all parameters θ and ϕ. However, we have a problem with the first term of the ELBO because we can not backpropagate through the sampling process of $\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})$, as we can see in the left part of Figure 3. To solve this problem we can use the **reparameterization trick** which consists in expressing the sampling process as a deterministic function of the parameters and some noise. For example, if we assume

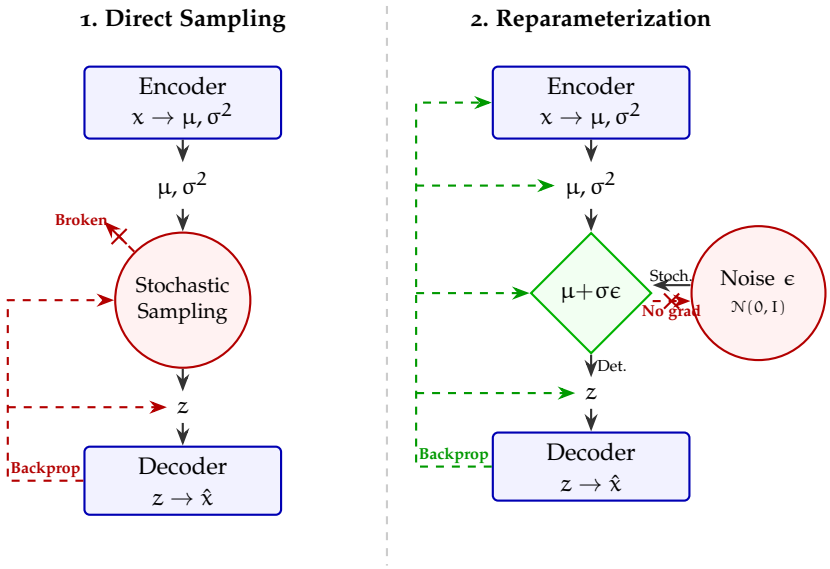


Figure 3: Reparameterization Trick in VAEs

that $q_\phi(\mathbf{z}|\mathbf{x})$ is a Gaussian distribution with mean μ and variance σ^2, we can sample from it as follows:

$$\mathbf{z} = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I). \tag{16}$$

This way, we can backpropagate through the deterministic function and optimize the parameters ϕ of the encoder. This is shown in the right part of Figure 3.

3.2 GAUSSIAN VAES

The common choice for both the encoder and the decoder distributions in VAEs is the Gaussian distribution. This is because it allows us to use the reparameterization trick and also because it has a closed-form expression for the KL divergence.
The **encoder** $q_\phi(\mathbf{z}|\mathbf{x})$ is modelled as a Gaussian distribution defined as:

$$q_\phi(\mathbf{z}|\mathbf{x}) = \mathcal{N}\left(\mathbf{z}; \mu_\phi(\mathbf{x}), \text{diag}(\sigma^2_\phi(\mathbf{x}))\right), \tag{17}$$

where $\mu_\phi(\mathbf{x})$ and $\sigma^2_\phi(\mathbf{x})$ are the mean and variance of the Gaussian distribution, which are parameterized by a neural network with parameters ϕ.
The **decoder** $p_\theta(\mathbf{x}|\mathbf{z})$ is also modelled as a Gaussian distribution defined as:

$$p_\theta(\mathbf{x}|\mathbf{z}) = \mathcal{N}\left(\mathbf{x}; \mu_\theta(\mathbf{z}), \sigma^2\mathbf{I}\right), \tag{18}$$

where $\mu_\theta(\mathbf{z})$ is the mean of the Gaussian distribution, which is parameterized by a neural network with parameters θ, and $\sigma^2\mathbf{I}$ is the

covariance matrix, which is usually fixed to be a diagonal matrix with constant variance.

Under this Gaussian assumption, we can obtain a closed-form expression for the ELBO loss function. The reconstruction term can be expressed as:

$$\mathbb{E}_{\mathbf{z}\sim q_\phi(\mathbf{z}|\mathbf{x})}\left[\log p_\theta(\mathbf{x}|\mathbf{z})\right] = -\frac{1}{2\sigma^2}\mathbb{E}_{\mathbf{z}\sim q_\phi(\mathbf{z}|\mathbf{x})}\left[\|\mathbf{x}-\mu_\theta(\mathbf{z})\|^2\right] + C, \quad (19)$$

where C is a constant that does not depend on the parameters. The KL divergence term can be expressed as:

$$\mathcal{D}_{KL}(q_\phi(\mathbf{z}|\mathbf{x})\|p(\mathbf{z})) = \frac{1}{2}\sum_{i=1}^{k}\left(\sigma^2_{\phi,i}(\mathbf{x}) + \mu^2_{\phi,i}(\mathbf{x}) - 1 - \log\sigma^2_{\phi,i}(\mathbf{x})\right), \quad (20)$$

where k is the dimension of the latent space. Therefore, the ELBO loss function can be expressed as:

$$\mathcal{L}_{ELBO}(\theta,\phi;\mathbf{x}) = -\frac{1}{2\sigma^2}\mathbb{E}_{\mathbf{z}\sim q_\phi(\mathbf{z}|\mathbf{x})}\left[\|\mathbf{x}-\mu_\theta(\mathbf{z})\|^2\right] \quad (21)$$

$$-\frac{1}{2}\sum_{i=1}^{k}\left(\sigma^2_{\phi,i}(\mathbf{x}) + \mu^2_{\phi,i}(\mathbf{x}) - 1 - \log\sigma^2_{\phi,i}(\mathbf{x})\right) + C. \quad (22)$$

Despite their mathematical elegance and continuous latent spaces, standard Gaussian Variational Autoencoders notoriously suffer from generating blurry images. This limitation fundamentally stems from the VAE objective function and its underlying statistical assumptions. The problem can be attributed to four primary factors:

- **MSE Loss**: In a standard Gaussian VAE with a fixed variance $\sigma^2\mathbf{I}$, maximizing the likelihood $p_\theta(\mathbf{x}|\mathbf{z})$ is mathematically equivalent to minimizing the $\mathcal{L}_2$ distance between the input $\mathbf{x}$ and the reconstruction $\hat{\mathbf{x}}$:

  $$\log p_\theta(\mathbf{x}|\mathbf{z}) \propto -\|\mathbf{x}-\mu_\theta(\mathbf{z})\|_2^2 \quad (23)$$

  The $\mathcal{L}_2$ loss heavily penalizes large pixel-wise deviations. When the model is uncertain about the exact location of high-frequency details (such as sharp edges), it outputs the statistical mean of all plausible variations to minimize the overall penalty. This phenomenon, known as *mode averaging*, results in safe, blurry predictions rather than rendering sharp, distinct features.

- **Information Bottleneck**: The ELBO includes a regularization term that minimizes the KL divergence between the approximate posterior and the prior:

  $$\mathcal{D}_{KL}(q_\phi(\mathbf{z}|\mathbf{x}) \parallel p(\mathbf{z})) \quad (24)$$

This term acts as an information bottleneck, forcing the encoded latent representations to closely match the simple prior, typically $\mathcal{N}(\mathbf{0}, \mathbf{I})$. Consequently, the encoder discards complex, high-frequency spatial features that are difficult to compress, prioritizing global structures and low-frequency components instead.

- **Posterior Collapse**: Furthermore, consistent with findings in [2] and [25], stochastic optimization using the unmodified lower bound objective frequently gets stuck in an undesirable stable equilibrium. At the onset of training, the reconstruction likelihood term $\log p_\theta(\mathbf{x}|\mathbf{z})$ is relatively weak. Consequently, an initially attractive state for the network is to aggressively minimize the latent cost, driving the approximate posterior to match the prior, $q_\phi(\mathbf{z}|\mathbf{x}) \approx p(\mathbf{z})$. This results in a stable equilibrium—often termed *posterior collapse*—from which the model struggles to escape, yielding uninformative latent representations and contributing to poor generation quality. To mitigate this, a common solution proposed by [2] and [25] is to implement an optimization schedule (KL annealing) where the weight of the latent cost $\mathcal{D}_{KL}(q_\phi(\mathbf{z}|\mathbf{x}) \parallel p(\mathbf{z}))$ is slowly annealed from 0 to 1 over multiple epochs.

3.3 DENOISING DIFFUSION PROBABILISTIC MODELS

One important technique developed to enhance the performance of VAEs and overcome the main limitations was **Hierarchical Variational Autoencoders** (HVAEs) [26] which are a type of VAE that uses a hierarchical structure in the latent space. This hierarchical structure allows the model to capture more complex and high-frequency details in the data, which can help to reduce the blurriness of the generated images. A visualization of the architecture of HVAEs is shown in Figure 4. In this architecture, we have L layers of latent variables $\mathbf{z}_1, \mathbf{z}_2, \ldots, \mathbf{z}_L$, where each layer captures different levels of abstraction.
In this setup, we can define the following factorization of the joint distribution:

$$p_\theta(\mathbf{x}, \mathbf{z}_{1:L}) = p_\theta(\mathbf{x}|\mathbf{z}_1) \prod_{i=2}^{L} p_\theta(\mathbf{z}_{i-1}|\mathbf{z}_i) p(\mathbf{z}_L), \tag{25}$$

and the marginal distribution of the data point $\mathbf{x}$ can be expressed as:

$$p_\theta(\mathbf{x}) = \int p_\theta(\mathbf{x}, \mathbf{z}_{1:L}) d\mathbf{z}_{1:L}, \tag{26}$$

where the generation is performed progressively from the latent variable $\mathbf{z}_L$, which follows the prio distribution $p(\mathbf{z}_L)$, to the data point $\mathbf{x}$,

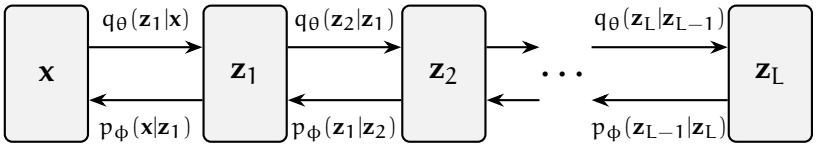


Figure 4: Hierarchical VAE architecture with L layers of latent variables. Each layer captures different levels of abstraction, allowing for richer representations and improved generation quality.

which is generated from the latent variable $\mathbf{z}_1$. The inference model can be defined as:

$$q_\phi(\mathbf{z}_{1:L}|\mathbf{x}) = q_\phi(\mathbf{z}_1|\mathbf{x}) \prod_{i=2}^{L} q_\phi(\mathbf{z}_i|\mathbf{z}_{i-1}), \tag{27}$$

where the inference is performed progressively from the data point $\mathbf{x}$ to the latent variable $\mathbf{z}_L$.
The ELBO loss is very similar to the one of the standard VAE, but we have to consider all the layers of latent variables:

$$\mathcal{L}_{ELBO}(\theta, \phi; \mathbf{x}) = \mathbb{E}_{\mathbf{z}_{1:L} \sim q_\phi(\mathbf{z}_{1:L}|\mathbf{x})} \left[\log \frac{p_\theta(\mathbf{x}, \mathbf{z}_{1:L})}{q_\phi(\mathbf{z}_{1:L}|\mathbf{x})} \right], \tag{28}$$

which can also be decomposed into the reconstruction term and the KL divergence term.

This method of deep, stacked layers revolutionized the field of generative modeling and is the basis for many state-of-the-art models. The training of HVAEs present several challenges because the encoder and decoder must be trained jointly and learning becamoes unstable. A clever twist to solve the main problems and maintain the hierarchical structure is the **Denoising Diffusion Probabilistic Models** (DDPMs) [8, 24], the next step in generative modeling.

Summary of Key Equations: VAEs and DDPMs

This box summarizes the main mathematical equations of VAEs and DDPMs:

1. Training Objective of VAEs:

$$\mathcal{L}_{\text{ELBO}}(\theta, \phi; \mathbf{x}) = \mathbb{E}_{\mathbf{z}\sim q_\phi(\mathbf{z}|\mathbf{x})}\left[\log p_\theta(\mathbf{x}|\mathbf{z})\right] - \mathcal{D}_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x})||p(\mathbf{z})). \tag{29}$$

2. Reparameterization Trick:

$$\mathbf{z} = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I). \tag{30}$$

3. Training Objective of Gaussian VAEs:

$$\mathcal{L}_{\text{ELBO}}(\theta, \phi; \mathbf{x}) = -\frac{1}{2\sigma^2}\mathbb{E}_{\mathbf{z}\sim q_\phi(\mathbf{z}|\mathbf{x})}\left[||\mathbf{x} - \mu_\theta(\mathbf{z})||^2\right] \tag{31}$$

$$-\frac{1}{2}\sum_{i=1}^{k}\left(\sigma^2_{\phi,i}(\mathbf{x}) + \mu^2_{\phi,i}(\mathbf{x}) - 1 - \log\sigma^2_{\phi,i}(\mathbf{x})\right) + C. \tag{32}$$

Note: Ensure latent dimensions z_i and z_j maintain near-zero mutual information to preserve explainability.

EBMs.

Summary of Key Equations: Energy-Based Models

This box summarizes the main mathematical equations of EBMs:

1. Training Objective of VAEs:

$$\mathcal{L}(\theta, \phi; \mathbf{x}) = \mathbb{E}_{z \sim q_\phi(\mathbf{z}|\mathbf{x})} [\log p_\theta(\mathbf{x}|\mathbf{z})] - D_{KL}(q_\phi(\mathbf{z}|\mathbf{x}) || p_\theta(\mathbf{z})) \quad (33)$$

2. Reparameterization Trick:

$$\mathbf{x}_{\text{synthetic}} = G(\mathbf{z}_{\text{anatomy}} \oplus \mathbf{z}_{\text{pathology}} \mid \mathbf{y}) \quad (34)$$

Note: Ensure latent dimensions z_i and z_j maintain near-zero mutual information to preserve explainability.

5 NORMALIZING FLOWS

Normalizing Flows.

6 DISENTANGLEMENT

Disentanglement Representation Learning (DRL) is a learning paradigm in which models are designed to extract and disentangle the hidden factors of variation in the data, $\mathbf{z} \in \mathbb{R}^c$, given an input observation $\mathbf{x} \in \mathbb{R}^d$. These generative factors are often called latent variables. The formal definition of DRL proposed in [1] is that "Disentangled representation should separate the distinct, independent and informative generative factors of variation in the data. Single latent variables are sensitive to changes in single underlying generative factors, while being relatively invariant to changes in other factors". An important property is that latent variables are statistically independent. This organisation of the latent space into explainable semantic representations improves the following properties of a generative model:

- **Explanability**: the meaningful and independent representations are aligned semantically with latent generative factors.
- **Generalizability**: the representations are well separated from the original entangled input, which improves the generalization ability for the generation of new samples.
- **Controllability**: DRL allows for controllable generation by manipulating the learned disentangled representations of the latent space.

There are several disentanglement methods in the current literature, which can be grouped based on the generative model they use [27], which will be explained in the following sections.
One important aspect of DRL is the

6.1 VAE-BASED MODELS

In Section – we defined the Variational Autoencoder (VAE) model, where the loss function is defined in Equation –. This type of model has the potential to learn disentangled representations in simple datasets, because of the choose of the prior distribution $p(\mathbf{z})$ as a normal distribution, which imposes independence constraints. In more complex datasets, the disentanglement capability presented in this model is poor.

One of the first methods was **β-VAE** [7], which introduces a β penalty

hyperparameter before the KL term in the ELBO loss. The new training objective is defined as

$$\mathcal{L}_{ELBO}(\theta, \phi; \mathbf{x}) = \mathbb{E}_{\mathbf{z}\sim q_\phi(\mathbf{z}|\mathbf{x})}\left[\log p_\theta(\mathbf{x}|\mathbf{z})\right] - \beta D_{KL}(q_\phi(\mathbf{z}|\mathbf{x})||p(\mathbf{z})), \quad (35)$$

where a value of $\beta = 1$ corresponds to the original VAE implementation and larger values of $\beta > 1$ encourage learning more disentangled representations, adding more pressure to the latent bottleneck to be more factorised, while reducing the performance of reconstruction. This shows an important trade-off between reconstruction accuracy and the disentanglement quality of the latent representation.

Several studies have analized this trade-off and the effect of the β hyperparameter in the disentanglement performance. First, we can define the marginal posterior distribution as

$$q_\phi(\mathbf{z}) = \mathbb{E}_{p_{data}(\mathbf{x})}\left[q_\phi(\mathbf{z}|\mathbf{x})\right] = \int q_\phi(\mathbf{z}|\mathbf{x}) p_{data}(\mathbf{x}) d\mathbf{x}, \quad (36)$$

where a disentangled representation is achieved when $q_\phi(\mathbf{z})$ is close to the factorial prior $p(\mathbf{z})$, which means we want a factorial distribution $q_\phi(\mathbf{z}) = \prod_j q_\phi(z_j)$. To gain a deep insight about the trade-off, we can break down the KL term of the loss function as proposed in several papers [4, 9]

$$\mathbb{E}_{p_{data}(\mathbf{x})}\left[KL(q_\phi(\mathbf{z}|\mathbf{x})||p(\mathbf{z}))\right] = \underbrace{I(\mathbf{x};\mathbf{z})}_{\text{(i) Mutual Information}} \quad (37)$$

$$+ \underbrace{D_{KL}(q_\phi(\mathbf{z})||\bar{q}_\phi(\mathbf{z}))}_{\text{(ii) Total Correlation}}, \quad (38)$$

$$+ \underbrace{D_{KL}(q_\phi(\mathbf{z})||p(\mathbf{z}))}_{\text{(iii) Prior KL}}, \quad (39)$$

where $I(\mathbf{x};\mathbf{z})$ is the mutual information between $\mathbf{x}$ and $\mathbf{z}$, Total Correlation (TC) is a measure of dependence between the variables and the prior KL prevents the latents to deviate too far from the prior. Here we can clearly see that making β larger pushes $q_\phi(\mathbf{z})$ towards the factorial prior $p(\mathbf{x})$ and pushes the model to find statistically independent factoes, improving the disentanglement, but reduces the amount of information about $\mathbf{x}$ stored in $\mathbf{z}$, producing a poor reconstruction.

This trade-off between reconstruction and disentanglement is the main motivation for the development of new methods based on the previous decomposition of the KL term. For example, the **DIP-VAE-I** and **DIP-VAE-II** [15] methods proposed a regularization term on the third

element of the KL decomposition. The objective function can be defined as

$$\mathcal{L}_{ELBO}(\theta, \phi; \mathbf{x}) = \mathbb{E}_{\mathbf{z}\sim q_\phi(\mathbf{z}|\mathbf{x})}\left[\log p_\theta(\mathbf{x}|\mathbf{z})\right] - D_{KL}(q_\phi(\mathbf{z}|\mathbf{x})||p(\mathbf{z})) \tag{40}$$

$$- \lambda D_{KL}(q_\phi(\mathbf{z})||p(\mathbf{z})). \tag{41}$$

This objective function imposes independence on the variational marginal distribution $q_\phi(\mathbf{z})$. To minimize the distance, they force the covariance of $q_\phi(\mathbf{z})$ to be close to the identity matrix. The covariance is defined as

$$\mathrm{Cov}_{q_\phi(\mathbf{z})}[\mathbf{z}] = \mathbb{E}_{p_{data}(\mathbf{x})}\left[\Sigma_\phi(\mathbf{x})\right] + \mathrm{Cov}_{p_{data}(\mathbf{x})}\left[\mu_\phi(\mathbf{x})\right]. \tag{42}$$

The two variations of DIP-VAE differ in the way they penalise the covariance:

$$D_{KL}(q_\phi(\mathbf{z})||p(\mathbf{z})) = \tag{43}$$

$$\underbrace{= \lambda_{od} \sum_{i\neq j} [\mathrm{Cov}_{p_{data}}[\mu_\phi(\mathbf{x})]]^2_{ij} + \lambda_d \sum_i \left(\mathrm{Cov}_{p_{data}}[\mu_\phi(\mathbf{x})]_{ij} - 1\right)^2}_{\text{DIP-VAE-I}}, \tag{44}$$

$$\text{or} \tag{45}$$

$$\underbrace{= \lambda_{od} \sum_{i\neq j} [\mathrm{Cov}_{q_\phi(\mathbf{z})}[\mathbf{z}]]^2_{ij} + \lambda_d \sum_i \left([\mathrm{Cov}_{q_\phi(\mathbf{z})}[\mathbf{z}]]_{ij} - 1\right)^2}_{\text{DIP-VAE-II}}, \tag{46}$$

where λ_{od} and λ_d are the hyperparameters that control the strength of the regularization.

The next method, **FactorVAE** [11], proposed to only penalises the second component of the KL decomposition, the Total Correlation (TC), which encourage independence in the latent space. The loss function in this method is defined as:

$$\mathcal{L}_{ELBO}(\theta, \phi; \mathbf{x}) = \mathbb{E}_{\mathbf{z}\sim q_\phi(\mathbf{z}|\mathbf{x})}\left[\log p_\theta(\mathbf{x}|\mathbf{z})\right] - D_{KL}(q_\phi(\mathbf{z}|\mathbf{x})||p(\mathbf{z})) \tag{47}$$

$$- \gamma D_{KL}(q_\phi(\mathbf{z})||\bar{q}_\phi(\mathbf{z})), \tag{48}$$

where $\bar{q}_\phi(\mathbf{z}) = \prod_j q_\phi(z_j)$. TC measures the dependencies between the variables in the latent representation. Penalising this specific term encourages the latent dimensions to be statistically independent, which is the core mathematical interpretation of disentanglement.
Because the Total Correlation is computationally intractable to evaluate directly over the entire dataset, FactorVAE employs an adversarial training approach. A discriminator network is trained to distinguish between samples drawn from the aggregated posterior $q_\phi(\mathbf{z})$

and samples drawn from the product of its marginals $\prod_j q_\phi(z_j)$, effectively estimating and minimizing the density ratio. This approach achieves better disentanglement than β-VAE while preserving a higher reconstruction quality.

The paper that proposed the decomposition of the KL term [4] also proposed a method called β-**TCVAE**, which penalize all the terms of the KL decomposition independently, with different hyperparameters. The loss function is defined as

$$\mathcal{L}_{\text{ELBO}}(\theta, \phi; \mathbf{x}) = \mathbb{E}_{\mathbf{z}\sim q_\phi(\mathbf{z}|\mathbf{x})}\left[\log p_\theta(\mathbf{x}|\mathbf{z})\right] \tag{49}$$

$$- \alpha I(\mathbf{x}; \mathbf{z}) - \beta D_{KL}(q_\phi(\mathbf{z})\|\bar{q}_\phi(\mathbf{z})) - \gamma D_{KL}(q_\phi(\mathbf{z})\|p(\mathbf{z})), \tag{50}$$

All the previous methods are unsupervised with the common idea of adding extra regularization terms to the original VAE loss function to encourage disentanglement.

6.2 DIFFUSION/FLOW-BASED MODELS

Diffusion and Flow models are a more recent and advanced class of generative models that have drawn a lot of attention in recent years because of their remarkable performance. Disentangled representation learning can also be applied to these types of models.

One of the first approaches was DisDiff [29], which learns a disentangled representation and a disentangled gradient field for each generative factor. Assume the data samples are generated by N underlying ground truth factors f^c, $c \in \{1, \dots, N\}$. Each factor c follows the distribution $p(f^c)$. We can modify the original score function (Equation –) with the conditioning of the factors using the Bayes' Rule as follows:

$$\nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t|f^c) = \nabla_{\mathbf{x}_t} \log p(f^c|\mathbf{x}_t) + \nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t), \tag{51}$$

where we already have the score $\nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t)$ from a pretrained unconditional model and we only have to learn $\nabla_{\mathbf{x}_t} \log p(f^c|\mathbf{x}_t)$. Since we are in an unsupervised setting, we do not know f^c. We have to learn a set of representations $\{z^c|c = 1, \dots, N\}$ that must encompass all information of the original samples $\mathbf{x}_0$ and represent the true factors of variations f^c. To obtain these representations we use a set of encoders with learnable parameters φ, defined as $E_\phi(\mathbf{x}_0) = \{E^1_\phi(\mathbf{x}_0), \dots E^N_\phi(\mathbf{x}_0)\} = \{z^1, \dots, z^N\}$. Then, assume we have a factor subset $S \subseteq C$ with $z^S = \{z^c|c \in S\}$, the gradient field can be defined as

$$\nabla_{\mathbf{x}_t} \log p(z^S|\mathbf{x}_t) = \sum_{c\in S} \nabla_{\mathbf{x}_t} \log p(z^c|\mathbf{x}_t). \tag{52}$$

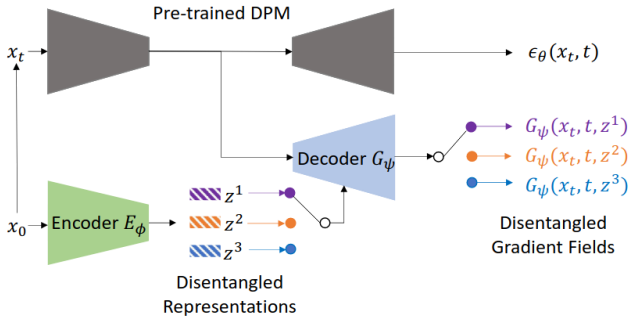
Finally, to estimate the gradient fields, they used a network $G_\psi(\mathbf{x}_t, z^c, t)$ and the reconstruction loss is defined as

$$\mathcal{L}_r = \mathbb{E}_{\mathbf{x}_0,t,\epsilon}\|\epsilon - \epsilon_\theta(\mathbf{x}_t, t) + \frac{\sqrt{\alpha_t}\sqrt{1-\bar{\alpha}_t}}{\beta_t}\sigma_t \sum_{c\in\mathcal{C}} G_\psi(\mathbf{x}_t, z^c, t)\|. \quad (53)$$

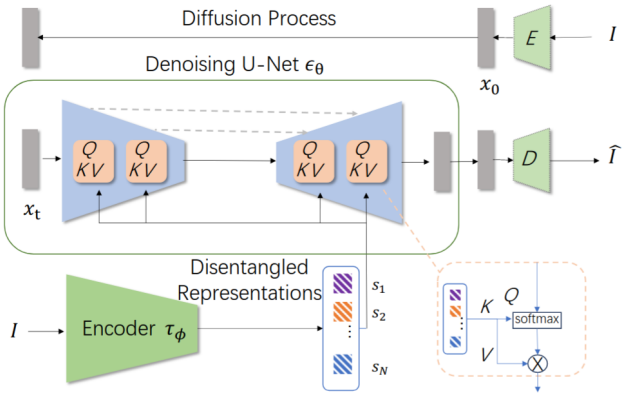
This loss formulation alone does not guarantee disentanglement among the representations z^c. The authors proposed a disentanglement loss that minimizes the upper bound for the mutual information between z^c and z^j, where $c \neq j$, defined as

$$\mathcal{L}_{dis} = \min_{i,j,x_0,\hat{x}_0^j} \|\hat{z}^{c|j} - \hat{z}^c\| - \|\hat{z}^{c|j} - z^c\| + \|\hat{z}^{j|j} - z^j\|, \quad (54)$$

where $\hat{\mathbf{x}}_0^j$ is conditioned on z^j, $\hat{z}^c = E_\phi^c(\hat{\mathbf{x}}_0)$ and $\hat{z}^{c|j} = E_\phi^c(\hat{\mathbf{x}}_0^j)$ where $\hat{\mathbf{x}}_0$ is a sampled data from the pretrained model. EncDiff [28] is an improvement of the previous method where an image encoder τ_ϕ aims to provide a set of concept tokens $\mathcal{S} = \{s_1, \ldots, s_N\}$. Each dimension of the encoded feature vectors is treated as a disentangled factor and maps each to a vector using a non-shared MLP layer. Then, based on Latent Diffusion Models [22], the idea is to use the cross-attention mechanism to map the tokens to the intermediate representations of



(a) DisDiff Framework

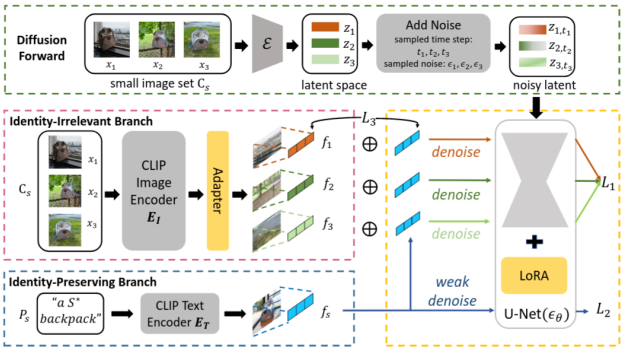


(b) EncDiff Framework

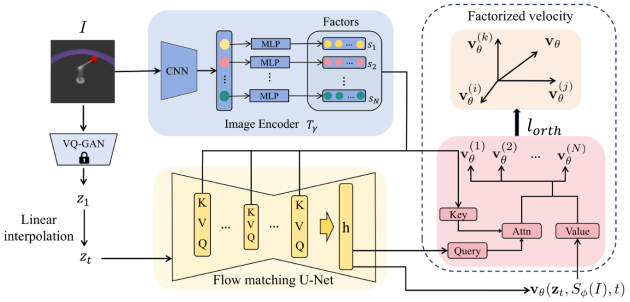
Figure 5: Illustrations of the frameworks for the (a) DisDiff model [29] and the (b) EncDiff model. [28]

the U-Net model. The general framework of this method is illustrated in Figure **??**.

DisenBooth [3] Flow Matching-Based disentanglement method [5]



(a) DisenBooth Framework



(b) FM Disentanglement Framework

Figure 6: Illustrations of the frameworks for the (a) DisenBooth model [3] and the (b) FM Disentanglement model. [5]

6.3 METRICS

7

COMPOSITIONALITY

Compositionality.

7.1 OBJECT-CENTRIC LEARNING

Slot Attention (SA) [17] is a differentiable interface between image features and a set of variables called slots. This method maps a set of N image patches with dimension D_{input}, called image features $\mathbf{F} \in \mathbb{R}^{N \times D_{input}}$, to a set of K output vectors of size D_{slot} called slots, $\mathbf{S} \in \mathbb{R}^{K \times D_{slot}}$. Each vector of the slots can describe an object or element in the input image. The process of binding a part of the image to the slots is performed via iterative refinement over T steps, where in each iteration the slots compete via a softmax attention mechanism to explain parts of the input. Slot representations are initialised randomly with a Gaussian distribution.
If we define some learnable linear transformations k, q and v to map input features and slots to a common dimension D, we can define:

$$\mathbf{A}_{i,j} = \frac{e^{\mathbf{M}_{i,j}}}{\sum_l e^{\mathbf{M}_{i,l}}}, \text{ where } \mathbf{M} = \frac{1}{\sqrt{D}} k(\mathbf{F}) \cdot q(\mathbf{S})^T \in \mathbb{R}^{N \times K} \tag{55}$$

The normalisation in the attention ensures that attention coefficients sum to one for each individual input feature vector. The update for the slots can be defined as:

$$\mathbf{U} = \mathbf{W}^T \cdot v(\mathbf{F}) \in \mathbb{R}^{K \times D} \text{ where } \mathbf{W}_{i,j} = \frac{\mathbf{A}_{i,j}}{\sum_{l=1}^{N} \mathbf{A}_{l,j}}. \tag{56}$$

Slot updates are performed via a Gated Recurrent Unit (GRU) and a small MLP layer with ReLU and residual connections, applied independently to each slot using shared parameters.
Once the slot representations are learned, they can be used for several downstream tasks. Some examples in the original paper [17] are

- **Object discovery**: SA can be used as a part of an autoencoder model for unsupervised object discovery. The encoder encodes the image in a set of slots, and the decoder reconstructs the original image. In this case, the slots act as a representation bottleneck. They proposed the use of a spatial broadcast decoder, which removes the upsampling deconvolutions. This new decoder first broadcasts the latent vector across the space (height and width of the image), appends fixed coordinate channels and applies a fully convolutional network to produce a final tensor with 4 channels, 3 for RGB and the alpha channel.

- **Set prediction**: In this scenario, we have an input image and a set of prediction targets. The main challenge is that the output order in SA is random, so there are K! possible equivalent representations for a set of K slots. In this case, they proposed to apply for each slot an MLP with shared parameters between slots. To overcome the problem of different prediction and label orders, they used a matching algorithm in the loss computation.

CODA [20]

8

CONCEPT BOTTLENECK MODELS

Concept Bottleneck Models (CBMs) were first introduced in [14] with the idea to easily interact with state-of-the-art models using high-level concepts that are understandable for the researchers. The general idea is to first predict an intermediate set of human-specified concepts **c** based on the input images and then use the concepts **c** to predict the target label **y**. The objective is to address three desiderata: [19]

- **Interpretability**: Being able to note which concepts are important for the targets.
- **Predictability**: Being able to predict the targets from the concepts alone.
- **Intervenability**: Being able to replace predicted concept values with ground truth values to improve predictive performance.

These types of models are trained on datapoints $\{(x^{(i)}, c^{(i)}, y^{(i)})\}_{i=1}^{n}$ where each input x (usually an image $x \in \mathbb{R}^{H\times W\times C}$) is annotated with the set of concepts $c \in \mathbb{R}^{k}$ with k concepts and the target $y \in \mathbb{R}$. We can define the bottleneck model as $f(g(x))$ where $g : \mathbb{R}^{H\times W\times C} \to \mathbb{R}^{k}$ maps an input x into the concept space and then $f : \mathbb{R}^{k} \to \mathbb{R}$ maps concepts into the target prediction. The bottleneck is performed when the target prediction $\hat{y} = f(\hat{c})$ relies only on the concept predictions $\hat{c} = g(x)$ based on the inputs. For the training of these models, we have three different strategies:

- **Independent**: learn $\hat{f}$ and $\hat{g}$ independently. For $\hat{f}$, minimise the task loss L_Y, which measures the discrepancy between predicted and true targets. Then, for $\hat{g}$, minimise the concept loss L_{C_j} for each concept j, which measures the discrepancy between predicted and real j-th concept.
- **Sequential**: first learn $\hat{g}$ and then use the predictions to learn $\hat{f}$.
- **Joint**: minimises the weighted sum of both task and concept losses with the hyperparameter λ, which controls the tradeoff between concept vs. task loss.

One important property of these models is the ability to perform interventions by editing the concept predictions $\hat{c}$ and propagating these changes to the target prediction $\hat{y}$. This property also makes these models more interpretable in terms of high-level concepts since we can see the importance and effect of the inclusion or removal of some

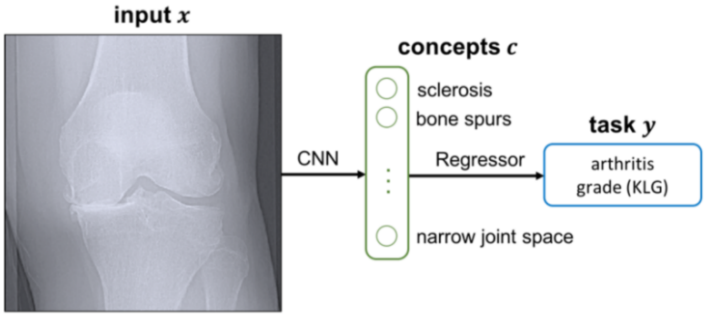


Figure 7: Caption

concepts for the final response. In the original paper, the concept intervention method is applied randomly without any specific criteria or using expert knowledge. In a recent paper [23], different intervention methods were evaluated to minimise the number of concepts to intervene on while maximising task accuracy. The method that achieves the best performance is the one where we first intervene on the concepts with the highest predictive uncertainty from the concept predictor g, i.e. it first intervenes on the concept with the maximum value of $1/|\hat{c}_i - 0.5|$.

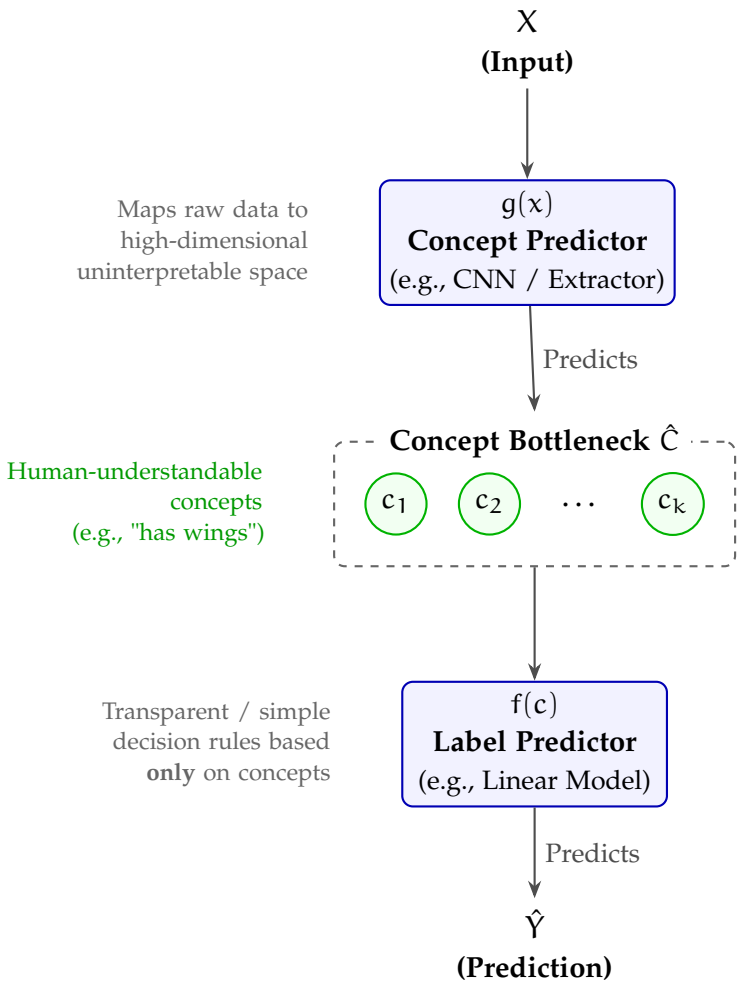


Figure 1: Architecture of a Concept Bottleneck Model (CBM). The model maps raw inputs X to an interpretable intermediate concept space Ĉ before making the final prediction Ŷ.

8.1 INFORMATION LEAKAGE

For these types of models, it may often be unrealistic to assume that we will have access to concepts which fully capture the relationship between the inputs and targets. Several recent papers have studied the problem of information leakage in these types of models based on the previous assumption. This information leakage is usually observed in CBMs trained sequentially or jointly [18, 19], and it appears when the intermediate representations encode information that is not present in the concepts c, which contains information to predict the final target rather than only focusing on getting the concept correct. This leakage has a negative impact on the interpretability and intervenability of the model. A CBM model with information leakage may

be unsafe for critical decision-making applications, such as clinical scenarios. [21] We can identify two main types of leakage:

- **Concept-task leakage**: additional information about the task is stored in the learnt concepts. If concept-task leakage is present, then the learned concept-task mutual information (MI) will be higher than ground-truth concept-task MI.
- **Interconcept leakage**: additional information about the other concepts is stored in each learned concept. Learned concepts are more predictive of the value of the other learned concepts, resulting in a learned interconcept MI being higher than the ground-truth.

Several measures of leakage have been proposed in the literature, but none of them has successfully integrated the information-theoretic nature of leakage in these types of models.

- **Concept Performance Metrics**: these are the metrics that indicate the quality of the concept learning, such as for example concept accuracy, precision, recall, F1 score and AUC. These metrics are not sensitive to leakage. Two models can have similar concept performance but have different degrees of information leakage. Concept AUC has more sensitivity in this problem since it contains more information about the concept distribution.
- **Oracle Impurity Score (OIS)**: this metric was defined in [6] to capture leakage. The idea is to train $k(k-1)$ additional simple MLP $\psi_{i,j}$, whose objective is to predict the value of the ground-truth concept j from the learnt representation of concept i. This will create the Purity Matrix, defined as $\pi(\hat{c}, c) = \mathrm{AUC}(\psi_{i,j}(\hat{c}_i), c_j)$. The (i,j)-th entry contains the AUC when predicting the ground truth value of concept j given the i-th concept representation. The OIS metric is the average difference between the predictivities of learnt and ground truth concepts over pairs of concepts, defined as

  $$\mathrm{OIS}(\hat{c}, c) = \frac{2}{k}\|\pi(\hat{c}, c) - \pi(c, c)\|_F$$

  , where the Frobenius norm ensures $0 \leqslant \mathrm{OIS} \leqslant 1$. The value of 1 represents a complete misalignment; the i-th concept can predict all other concepts except its corresponding one. A value of 0 represents a perfect alignment; the i-th concept does not encode any unnecessary information for predicting concept i. A limitation is the stochasticity of the metric, which involves training neural networks, which is very variable when we evaluate the model several times at test time.

- **Niche Impurity Score (NIS)**:
- **Intervention**: the intervention of the predicted concepts with the ground-truth values can be used to evaluate the interpretability and the leakage. A task accuracy that decreases after intervention indicates that the task head expects extra information that is not present in the ground-truth concepts. Models with higher leakage have worse intervention performance. This metric is unable to distinguish between interconcept and concept-task leakage; also, when we have a large number of concepts, the computation is very expensive.

8.2 RECENT CBMS

8.3 CONCEPT BOTTLENECK GENERATIVE MODELS

All the previous models presented in this document are focused on classification or regression problems, which are the original implementations of concept bottleneck models. The next step is to combine this idea with generative models to develop a method for interpreting and intervening in them, called Concept Bottleneck Generative Models (CBGMs)[10]. This method allows us to steer the generative model, since we can change the concept of the image while preserving the image quality, and understand the model by identifying the important concepts that are responsible for the output.
The general idea is to insert a concept bottleneck (CB) layer into a generative model, which is divided in two parts: the pre-concept bottleneck part and the post-concept bottleneck part. The partition depends on the generative model we use (GANs, VAEs, Diffusion). The authors decided to use the Concept Embedding framework [30] for the concept bottleneck layer and also argue that it is unrealistic to expect that the pre-defined human concepts are complete, so they also added a set of unknown concepts that might also be required for generation.
Each concept is represented with two embeddings: $w_i^+, w_i^- \in \mathbb{R}^m$ representing the active and inactive concept states, respectively. The output of the pre-concept bottleneck part of the model, the embedding h, is fed into several context networks ϕ that map h to the two embeddings per concept. Then, a network Ψ is trained to predict the probability of concept c_i from the joint embedding space, $\hat{c}_i = \Psi_i([w_i^+, w_i^-]^T) \in [0, 1]$. The final context embedding w_i is defined as a weighted mixture of the two embeddings as $w_i = (\hat{c}_i w_i^+ + (1 - \hat{c}_i) w_i^-)$. All the k concept embeddings are concatenated to obtain the final vector that is fed to the post-concept network. For the intervention, we simply have to modify the concept probability $\hat{c}_i$ to $\bar{c}_i$ in

the weighted mixture of w_i.
The loss function is defined as

$$\mathcal{L} = \mathcal{L}_{task} + \alpha \mathcal{L}_{con} + \beta \mathcal{L}_{orth},$$

where α and β are hyperparameters that control the importance of the concept and orthogonality loss, respectively. As we can see, the loss consists of three parts:

- **Task Loss**: the base objective function of the generative model family.
- **Concept Loss**: is the binary cross-entropy loss between the predicted and real concept probabilities.
- **Concept Orthogonality Loss**: encourage the unknown concepts to be orthogonal to the outputs of each concept network using an orthogonality constraint defined as:

$$\mathcal{L}_{orth} = \sum_{f \in B} \frac{\sum_{i=1}^{i=k} |\langle w_i, w_{k+1} \rangle|}{\sum_{i=1}^{i=k} 1},$$

  where $\langle \cdot, \cdot \rangle$ is the cosine similarity and j denotes each sample in the mini-batch of size B.

Part II

METHODS

9 PROPOSAL

Proposal.

10 DATA

In this section I want to introduce the different datasets that I have used during this part of my research and the possible future datasets that I plan to use in the remaining part of my thesis. In this initial part where I have focused on the implementation of some methods found in the literature along with some small experiments to have an intuition about the field, I have used a collection of small and well known in the literature datasets. These datasets are the following:

- **MNIST** [**lecun1998mnist**]: a dataset of handwritten digits with 60,000 training samples and 10,000 test samples, where each sample is a 28x28 grayscale image of a digit from 0 to 9.
- **Shapes2D** [**locatello2019challenging**]: a dataset of 2D shapes with 737,280 samples, where each sample is a 64x64 RGB image of a shape (square, ellipse or heart) with different colors and sizes.
- **Shapes3D** [**locatello2019challenging**]: a dataset of 3D shapes with 480,000 samples, where each sample is a 64x64 RGB image of a 3D shape (cube, sphere or cylinder) with different colors and sizes.
- **dSprites** [**dsprites17**]: a dataset of 2D sprites with 737,280 samples, where each sample is a 64x64 grayscale image of a sprite with different shapes, colors and positions.
- **CelebA** [**liu2015faceattributes**]: a dataset of celebrity faces with 202,599 samples, where each sample is a 178x218 RGB image of a celebrity face with different attributes (gender, age, hair color, etc.).
- **CelebA-HQ** [**karras2017progressive**]: a dataset of high-quality celebrity faces with 30,000 samples, where each sample is a 1024x1024 RGB image of a celebrity face with different attributes (gender, age, hair color, etc.).

Part III

EXPERIMENTS AND RESULTS

11

UNCONDITIONAL FLOW MATCHING MODEL FOR X-RAY IMAGES

Unconditional Flow Matching Model for X-Ray Images.

12

SLOT ATTENTION WITH CONCEPT SUPERVISION IN CELEBA-HQ

After the literature review about Slot Attention and other Object-Centric Learning methods, where all the explanations can be found in Section –, some connections with Concept Bottleneck Models (Section –) arise naturally. Both paradigms inherently seek to decompose complex, high-dimensional visual inputs into a discrete set of interpretable variables before performing downstream reasoning. In both frameworks, the architecture forces the model to route its decision-making process through a constrained intermediate layer—whether it is a set of permutation-invariant slots representing physical entities, or a vector of semantic concepts aligned with human logic.

Despite this shared structural philosophy, there are fundamental differences in their objectives and learning signals. Object-Centric Learning methods like Slot Attention are traditionally unsupervised, relying on spatial competition and reconstruction objectives to discover emergent, perceptual groupings. In contrast, Concept Bottleneck Models are explicitly supervised, requiring ground-truth labels to map visual features directly to predefined, high-level semantics. Furthermore, while traditional CBMs often struggle to spatially ground their concepts within specific regions of an image, Slot Attention excels at isolating localized areas but lacks the guarantee that a discovered slot will align with a human-understandable trait.

Recognizing both the complementary strengths and the distinct differences between these approaches, this section introduces an experiment that bridges the two. By applying explicit concept supervision to a Slot Attention module, we aim to spatially ground predefined facial attributes from the CelebA-HQ dataset directly into distinct visual slots. To ensure the slots can specialize entirely on these localized semantic traits, our architecture incorporates register tokens designed to independently capture and store extra global image context.

Part IV

RESEARCH PLAN

Part V

APPENDIX

APPENDIX TEST

Lorem ipsum at nusquam appellantur his, ut eos erant homero concludaturque. Albucius appellantur deterruisset id eam, vivendum partiendo dissentiet ei ius. Vis melius facilisis ea, sea id convenire referrentur, takimata adolescens ex duo. Ei harum argumentum per. Eam vidit exerci appetere ad, ut vel zzril intellegam interpretaris.

More dummy text.

A.1 APPENDIX SECTION TEST

Test: Table 1 (This reference should have a lowercase, small caps A if the option `floatperchapter` is activated, just as in the table itself → however, this does not work at the moment.)

LABITUR BONORUM PRI NO	QUE VISTA	HUMAN
fastidii ea ius	germano	demonstratea
suscipit instructior	titulo	personas
quaestio philosophia	facto	demonstrated

Table 1: Autem usu id.

A.2 ANOTHER APPENDIX SECTION TEST

Equidem detraxit cu nam, vix eu delenit periculis. Eos ut vero constituto, no vidit propriae complectitur sea. Diceret nonummy in has, no qui eligendi recteque consetetur. Mel eu dictas suscipiantur, et sed placerat oporteat. At ipsum electram mei, ad aeque atomorum mea. There is also a useless Pascal listing below: Listing 1.

Listing 1: A floating example (`listings` manual)

```
for i:=maxint downto 0 do
begin
{ do nothing }
end;
```

BIBLIOGRAPHY

[1] Yoshua Bengio, Aaron Courville, and Pascal Vincent. "Representation Learning: A Review and New Perspectives." In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 35.8 (2013), pp. 1798–1828. DOI: 10.1109/TPAMI.2013.50.

[2] Samuel R. Bowman, Luke Vilnis, Oriol Vinyals, Andrew M. Dai, Rafal Jozefowicz, and Samy Bengio. *Generating Sentences from a Continuous Space*. 2016. arXiv: 1511.06349 [cs.LG]. URL: https://arxiv.org/abs/1511.06349.

[3] Hong Chen, Yipeng Zhang, Simin Wu, Xin Wang, Xuguang Duan, Yuwei Zhou, and Wenwu Zhu. *DisenBooth: Identity-Preserving Disentangled Tuning for Subject-Driven Text-to-Image Generation*. 2024. arXiv: 2305.03374 [cs.CV]. URL: https://arxiv.org/abs/2305.03374.

[4] Ricky T. Q. Chen, Xuechen Li, Roger Grosse, and David Duvenaud. *Isolating Sources of Disentanglement in Variational Autoencoders*. 2019. arXiv: 1802.04942 [cs.LG]. URL: https://arxiv.org/abs/1802.04942.

[5] Jinjin Chi, Taoping Liu, Mengtao Yin, Ximing Li, Yongcheng Jing, Jialie Shen, Leszek Rutkowski, and Dacheng Tao. *Disentangled Representation Learning via Flow Matching*. 2026. arXiv: 2602.05214 [cs.LG]. URL: https://arxiv.org/abs/2602.05214.

[6] Mateo Espinosa Zarlenga, Pietro Barbiero, Zohreh Shams, Dmitry Kazhdan, Umang Bhatt, Adrian Weller, and Mateja Jamnik. "Towards Robust Metrics for Concept Representation Evaluation." In: *Proceedings of the AAAI Conference on Artificial Intelligence* 37.10 (2023), 11791–11799. ISSN: 2159-5399. DOI: 10.1609/aaai.v37i10.26392. URL: http://dx.doi.org/10.1609/aaai.v37i10.26392.

[7] Irina Higgins, Loic Matthey, Arka Pal, Christopher Burgess, Xavier Glorot, Matthew Botvinick, Shakir Mohamed, and Alexander Lerchner. "beta-VAE: Learning Basic Visual Concepts with a Constrained Variational Framework." In: *International Conference on Learning Representations*. 2017. URL: https://openreview.net/forum?id=Sy2fzU9gl.

[8] Jonathan Ho, Ajay Jain, and Pieter Abbeel. *Denoising Diffusion Probabilistic Models*. 2020. arXiv: 2006.11239 [cs.LG]. URL: https://arxiv.org/abs/2006.11239.

[9] Matthew D Hoffman and Matthew J Johnson. "ELBO surgery: yet another way to carve up the variational evidence lower bound." In: *NIPS Workshop on Advances in Approximate Bayesian Inference*. 2016.

[10] Aya Abdelsalam Ismail, Julius Adebayo, Hector Corrada Bravo, Stephen Ra, and Kyunghyun Cho. "Concept Bottleneck Generative Models." In: *The Twelfth International Conference on Learning Representations*. 2024. URL: https://openreview.net/forum?id=L9U5MJJleF.

[11] Hyunjik Kim and Andriy Mnih. *Disentangling by Factorising*. 2019. arXiv: 1802.05983 [stat.ML]. URL: https://arxiv.org/abs/1802.05983.

[12] Diederik P. Kingma and Max Welling. "An Introduction to Variational Autoencoders." In: *Foundations and Trends® in Machine Learning* 12.4 (Nov. 2019), 307–392. ISSN: 1935-8245. DOI: 10.1561/2200000056. URL: http://dx.doi.org/10.1561/2200000056.

[13] Diederik P Kingma and Max Welling. *Auto-Encoding Variational Bayes*. 2022. arXiv: 1312.6114 [stat.ML]. URL: https://arxiv.org/abs/1312.6114.

[14] Pang Wei Koh, Thao Nguyen, Yew Siang Tang, Stephen Mussmann, Emma Pierson, Been Kim, and Percy Liang. *Concept Bottleneck Models*. 2020. arXiv: 2007.04612 [cs.LG]. URL: https://arxiv.org/abs/2007.04612.

[15] Abhishek Kumar, Prasanna Sattigeri, and Avinash Balakrishnan. *Variational Inference of Disentangled Latent Concepts from Unlabeled Observations*. 2018. arXiv: 1711.00848 [cs.LG]. URL: https://arxiv.org/abs/1711.00848.

[16] Chieh-Hsin Lai, Yang Song, Dongjun Kim, Yuki Mitsufuji, and Stefano Ermon. *The Principles of Diffusion Models*. 2025. arXiv: 2510.21890 [cs.LG]. URL: https://arxiv.org/abs/2510.21890.

[17] Francesco Locatello, Dirk Weissenborn, Thomas Unterthiner, Aravindh Mahendran, Georg Heigold, Jakob Uszkoreit, Alexey Dosovitskiy, and Thomas Kipf. *Object-Centric Learning with Slot Attention*. 2020. arXiv: 2006.15055 [cs.LG]. URL: https://arxiv.org/abs/2006.15055.

[18] Anita Mahinpei, Justin Clark, Isaac Lage, Finale Doshi-Velez, and Weiwei Pan. *Promises and Pitfalls of Black-Box Concept Learning Models*. 2021. arXiv: 2106.13314 [cs.LG]. URL: https://arxiv.org/abs/2106.13314.

[19] Andrei Margeloiu, Matthew Ashman, Umang Bhatt, Yanzhi Chen, Mateja Jamnik, and Adrian Weller. *Do Concept Bottleneck Models Learn as Intended?* 2021. arXiv: 2105.04289 [cs.LG]. URL: https://arxiv.org/abs/2105.04289.

[20] Bac Nguyen, Yuhta Takida, Naoki Murata, Chieh-Hsin Lai, Toshimitsu Uesaka, Stefano Ermon, and Yuki Mitsufuji. *Improved Object-Centric Diffusion Learning with Registers and Contrastive Alignment*. 2026. arXiv: 2601.01224 [cs.CV]. URL: https://arxiv.org/abs/2601.01224.

[21] Enrico Parisini, Tapabrata Chakraborti, Chris Harbron, Ben D. MacArthur, and Christopher R. S. Banerji. *Leakage and Interpretability in Concept-Based Models*. 2026. arXiv: 2504.14094 [cs.LG]. URL: https://arxiv.org/abs/2504.14094.

[22] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. *High-Resolution Image Synthesis with Latent Diffusion Models*. 2022. arXiv: 2112.10752 [cs.CV]. URL: https://arxiv.org/abs/2112.10752.

[23] Sungbin Shin, Yohan Jo, Sungsoo Ahn, and Namhoon Lee. "A Closer Look at the Intervention Procedure of Concept Bottleneck Models." In: *Workshop on Trustworthy and Socially Responsible Machine Learning, NeurIPS 2022*. 2022. URL: https://openreview.net/forum?id=PUspzfGsgY.

[24] Jascha Sohl-Dickstein, Eric A. Weiss, Niru Maheswaranathan, and Surya Ganguli. *Deep Unsupervised Learning using Nonequilibrium Thermodynamics*. 2015. arXiv: 1503.03585 [cs.LG]. URL: https://arxiv.org/abs/1503.03585.

[25] {Casper Kaae} Sønderby, Tapani Raiko, Lars Maaløe, {Søren Kaae} Sønderby, and Ole Winther. "How to Train Deep Variational Autoencoders and Probabilistic Ladder Networks." English. In: *Proceedings of the 33rd International Conference on Machine Learning (ICML 2016)*. JMLR: Workshop and Conference Proceedings. 33rd International Conference on Machine Learning (ICML 2016), ICML ; Conference date: 19-06-2016 Through 24-06-2016. 2016. URL: http://icml.cc/2016/.

[26] Arash Vahdat and Jan Kautz. *NVAE: A Deep Hierarchical Variational Autoencoder*. 2021. arXiv: 2007.03898 [stat.ML]. URL: https://arxiv.org/abs/2007.03898.

[27] Xin Wang, Hong Chen, Si'ao Tang, Zihao Wu, and Wenwu Zhu. *Disentangled Representation Learning*. 2024. arXiv: 2211.11695 [cs.LG]. URL: https://arxiv.org/abs/2211.11695.

[28] Tao Yang, Cuiling Lan, Yan Lu, and Nanning zheng. *Diffusion Model with Cross Attention as an Inductive Bias for Disentanglement*. 2024. arXiv: 2402.09712 [cs.CV]. URL: https://arxiv.org/abs/2402.09712.

[29] Tao Yang, Yuwang Wang, Yan Lv, and Nanning Zheng. *DisDiff: Unsupervised Disentanglement of Diffusion Probabilistic Models*. 2023. arXiv: 2301.13721 [cs.CV]. URL: https://arxiv.org/abs/2301.13721.

[30] Mateo Espinosa Zarlenga et al. *Concept Embedding Models: Beyond the Accuracy-Explainability Trade-Off*. 2022. arXiv: 2209.09056 [cs.LG]. URL: https://arxiv.org/abs/2209.09056.